VAL CoPilot

Azure AI Foundry Deployment Guide

Optional future path — current POC runs Streamlit → Flask → FastMCP → synthetic Gold

Part A = Azure Foundry portal steps | Part B = GitHub repo / deploy script

*Complete **Part A** fully before running* `deploy_to_foundry.py`.

Framing:** Azure AI Foundry is an **optional future host**, not the current validated POC runtime. The Synthetic Contract Intelligence Tool-Layer POC runs locally as **Streamlit → Flask cognitive router → FastMCP tools → LinkSquares offline fixtures (*`USE_OFFLINE_MOCKS=true`), with** SQLite persona memory** (save / recall previous searches) and a **compare hard-stop** when suppliers or contract IDs cannot be resolved. Use this guide only when you intentionally want to wrap the MCP tool surface in a managed Foundry Agent.*

Step-by-step procedure to provision VAL CoPilot into Microsoft Azure AI Foundry using `copilot_agent/deploy_to_foundry.py`.

Audience: Azure Cloud Architects / DevSecOps Engineers

SDKs: `azure-ai-projects==1.0.0` · `azure-ai-agents` · `azure-identity`

This guide is split into two parts:

Part	Where you work	Outcome
A — Azure Foundry side	Azure Portal + ai.azure.com	Project, model, RBAC, endpoint ready
B — Code / script side	Developer machine with the GitHub repo	Managed agent created with SYSTEM_PROMPT + tools

Do Part A completely before Part B. The deploy script cannot create the Foundry resource, project, or model deployment for you.

---

1. Overview

This guide optionally deploys VAL CoPilot as a **managed Azure AI Foundry Agent** that hosts the same cognitive instructions and MCP tool surface used by the local tool-layer POC. The deployment script authenticates with Entra ID, wraps MCP tool implementations in a Foundry **FunctionTool / ToolSet**, and creates the agent using **SYSTEM_PROMPT** (lifecycle procedures, invoice/spend OOS guardrail, compare hard-stop, persona-memory guidance, comparative analysis → `## Recommendation` when compare succeeds).

Current POC vs this Foundry path

Path	Host	Data	When to use
Current POC (default)	Streamlit + Flask + FastMCP + persona memory	LinkSquares offline fixtures	Demos, local validation, CI guards

Optional Foundry	Managed Foundry Agent + ToolSet (subset today)	Fixtures or Fabric/Search when configured	Future Teams / Foundry hosting

What gets provisioned (by the script)

• Managed agent named `val-copilot` (configurable)
• Model deployment referenced by `AZURE_OPENAI_DEPLOYMENT_NAME` or `AZURE_FOUNDRY_MODEL_DEPLOYMENT`
• ToolSet containing **7** structured contract-intelligence MCP tools (local FastMCP also exposes `compare_contracts` and `check_missing_contract_fields` — extend the ToolSet next):
- `search_contracts`
- `get_contract_profile`
- `get_expiring_contracts`
- `get_vendor_spend_summary` (committed-value rollup — **not** invoice actuals)
- `find_overlaps`
- `explain_contract_risk`
- `search_cloud_blob_contracts`
• Instructions = full `SYSTEM_PROMPT` from `copilot_agent/agent.py`, including:
- Hard OOS message for invoice/spend-actuals questions
- Compare hard-stop when any requested supplier/ID is unresolved (exact message only; no default `CON-0001` vs `CON-0002` pair)
- Persona-memory guidance (local SQLite recall remains on the Flask/Streamlit path today)

What you must prepare first (on Azure)

• Foundry account / resource
• Foundry **project** (new Foundry project — not a classic hub-only workspace)
• At least one **chat model deployment** in that project
• Entra ID **RBAC** for the identity that runs the script
• Project **endpoint** URL copied into root `.env`
• (For live tool backends) Fabric SQL and/or Azure AI Search reachable by the execution identity — **not required** if tools run against offline fixtures

---

PART A — Azure AI Foundry side (complete these first)

Work in this order. Each step ends with a **Done when** check so you know you can move on.

A1. Sign in and choose the right portal experience

1. Open https://ai.azure.com (Microsoft Foundry portal).
2. Sign in with an Azure account that can create resources in your target subscription.
3. If you see a **New Foundry** toggle, turn it **ON**. These steps refer to the current Foundry experience (not Foundry classic / hub-only).
4. Confirm the correct **Directory** and **Subscription** in the portal account menu.

Done when: You are on the Foundry home page for the intended subscription.

A2. Create the Foundry resource and project

You need a **Foundry project** under a Foundry / AI Services account. Prefer a **foundry-based project** (project endpoint visible on the overview). Classic hub-only projects often show only a connection string and may not work with this SDK path.

Option 1 — Portal (recommended)

1. In Foundry, choose **Create a new project** (or **Create** → Foundry / project).
2. Enter a **project name** (example: `val-copilot-proj`).
3. Open **Advanced** options if shown and set:
- **Subscription**
- **Resource group** (create new or reuse)
- **Region** that supports **Agents** (examples commonly used: East US, West Europe, Southeast Asia — confirm Agents availability for your region)
- Foundry / AI Services **resource name** if prompted
4. Create and wait until provisioning finishes (often several minutes).
5. Open the new project when ready.

Option 2 — Azure Portal

1. Azure Portal → **Create a resource** → search for **Azure AI Foundry** / **AI Services** (Foundry).
2. Create the account with a region that supports Agents.
3. From the account, create / open a **Project**.
4. Open the project in ai.azure.com.

Done when: Project Home / Overview loads and shows a **Project endpoint** (URL), not only a legacy hub connection string.

A3. Deploy a chat model in the project

The managed agent needs a **deployment name** that already exists in the project.

1. In the Foundry project, open **Discover** → **Models** (or **Deployments** / **Models + endpoints**, depending on UI).
2. Select a chat model suitable for tool-calling agents (example: `gpt-4o` or another GPT-4-class model available to your subscription).
3. Click **Deploy**.
4. Prefer **Custom settings** if you need a specific deployment name; otherwise **Default settings** is fine.
5. Record the **Deployment name** exactly (case-sensitive). Example: `gpt-4o`.
6. Wait until the deployment status is **Succeeded** / Ready.
7. Optional smoke test: open the model playground and send “hello” to confirm inference works.

Done when: Deployments list shows your model as ready, and you have copied the exact **deployment name**.

A4. Confirm Agents capability is available

1. In the project left navigation, open **Build** → **Agents** (or **Agents**).
2. Confirm the Agents blade loads (empty list is OK before the script runs).
3. If Agents is missing or blocked:
- Switch to a **supported region**
- Ensure you are on **New Foundry** (not classic hub-only)
- Check subscription policy / quota with your Azure admin

Done when: You can open the Agents page for this project without an error.

A5. Assign Entra ID roles (RBAC)

The deploy script uses **DefaultAzureCredential** (your user, service principal, or managed identity). That identity must be allowed to create agents on the project.

1. Azure Portal → open the **Foundry account** or **project** resource.

2. Open **Access control (IAM)** → **Add role assignment**.
3. Assign at least one of:
- **Azure AI Developer** (typical for agent create / run), or
- **Azure AI Project Manager** / owner-equivalent if your org requires it
4. Assign to the identity that will run `deploy_to_foundry.py` (your user UPN or the service principal).
5. If teammates will use the agent, grant them the same (or Reader + run permissions per your org standard).
Also plan data-plane access (needed when tools actually run against live backends).

For fixture-backed demos, set `USE_OFFLINE_MOCKS=true` on the tool-host process instead.

Data system	Typical need
Synthetic fixtures (POC)	`USE_OFFLINE_MOCKS=true` + LinkSquares sample JSON in `mcp_server/`
Microsoft Fabric SQL (optional)	Identity can connect via `ActiveDirectoryDefault` to the Gold warehouse
Azure AI Search (optional)	Search index reader (or API key in `.env` for key-based access)

Done when: The deploying identity has Foundry project rights, and a tool data path (fixtures or Fabric/Search) is planned.

A6. Copy the Project endpoint (required for `.env`)

1. Open the Foundry project **Home / Overview / Welcome** page.
2. Find **Project endpoint** (copy button).
3. It should look like:

```
https://<ai-services-account-name>.services.ai.azure.com/api/projects/<project-name>
```

4. Paste this value into root `.env` as `AZURE_FOUNDRY_CONNECTION_STRING` (Part B).

Acceptable alternate form

Legacy semicolon connection string is also accepted by the script if it includes `endpoint=`:

```
endpoint=https://<account>.services.ai.azure.com/api/projects/<project>;subscriptionId=<guid>;resourceGroupName=<rg
```

Warning — hub-only projects

If the portal only shows a classic **connection string** and **no** …`/api/projects/`… endpoint, you are likely on a hub-based project. Create a **new Foundry project** with a project endpoint before continuing.

Done when: You have saved the project endpoint URL and the model deployment name.

A7. Networking / firewall (if your tenant locks egress)

Complete this on Azure if corporate firewalls or private endpoints are enabled:

1. Foundry / Azure OpenAI networking blade → allow your runner's public IPs **or** private endpoint path.
2. Azure AI Search networking → allow the same runner / VNet.
3. Fabric warehouse → allow the identity and network path used by ODBC.
Done when: A test call from the runner machine can reach Foundry (and later Fabric/Search).

A8. Azure Foundry readiness checklist (gate to Part B)

Do **not** run the Python deploy script until every box is checked:

- ■ New Foundry portal experience enabled (not classic-only)
- ■ Foundry account created in an Agents-supported region
- ■ Foundry **project** created and openable in ai.azure.com

- ■ **Project endpoint** URL copied (`https://….services.ai.azure.com/api/projects/…`)
- ■ Chat **model deployed** and status = Ready
- ■ Exact **deployment name** recorded (example: `gpt-4o`)
- ■ **Agents** blade opens successfully
- ■ Deploying identity has **Azure AI Developer** (or equivalent) on the project
- ■ Tool data path planned (offline fixtures **or** Fabric SQL / Azure AI Search)
- ■ Networking allowlists completed (if required for live backends)

PART B — Code / script side (after Part A is complete)

B1. Clone the GitHub repository

```
git clone https://github.com/booshank/VALAgent.git
cd VALAgent
# use the branch that contains deploy_to_foundry.py if not yet on main
git checkout cursor/val-copilot-workspace-a9d1
```

B2. Configure root `.env`

Copy `.env.example` to `.env` at the repository root (do **not** create per-folder env files).

Map values collected in Part A:

Variable	Required	Source (Azure side)
`AZURE_FOUNDRY_CONNECTION_STRING`	Yes	Project endpoint from **A6**
`AZURE_FOUNDRY_MODEL_DEPLOYMENT` or `AZURE_OPENAI_DEPLOYMENT_NAME`	Yes	Exact deployment name from **A3**
`AZURE_FOUNDRY_AGENT_NAME`	No	Desired agent name (default `val-copilot`)
`USE_OFFLINE_MOCKS`	POC demos	`true` to use LinkSquares sample fixtures instead of live Fabric
`FABRIC_SQL_SERVER` / `FABRIC_SQL_DATABASE`	Live runtime	Fabric warehouse (optional)
`AZURE_SEARCH_ENDPOINT` / API key / index	Live runtime	Azure AI Search (optional)

Example:

```
AZURE_FOUNDRY_CONNECTION_STRING=https://myacct.services.ai.azure.com/api/projects/val-copilot-proj
AZURE_FOUNDRY_MODEL_DEPLOYMENT=gpt-4o
AZURE_FOUNDRY_AGENT_NAME=val-copilot
AZURE_OPENAI_DEPLOYMENT_NAME=gpt-4o
```

B3. Authenticate (`DefaultAzureCredential`)

No Foundry API key is used by the deploy script for control-plane auth.

1. **Developer workstation:** `az login` then `az account set --subscription <id>`
2. **Service principal:** set `AZURE_CLIENT_ID`, `AZURE_TENANT_ID`, `AZURE_CLIENT_SECRET`
3. **CI / VM:** managed identity with the RBAC from **A5**

```
az login
az account show --query id -o tsv
```

B4. Install Python dependencies

```
cd copilot_agent
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
pip install -r ../mcp_server/requirements.txt
```

Pinned packages: `azure-ai-projects==1.0.0`, `azure-ai-agents>=1.0.0,<2.0.0`,

`azure-identity>=1.19.0`. Do **not** upgrade to `azure-ai-projects` 2.x for this

script — `ToolSet` was removed.

B5. Dry-run (local validation only)

```
cd copilot_agent
python deploy_to_foundry.py --dry-run -v
```

Expected:

• `status: dry_run`
• tools include: `search_contracts`, `get_contract_profile`, `get_expiring_contracts`, `get_vendor_spend_summary`, `find_overlaps`, `explain_contract_risk`, `search_cloud_blob_contracts`
• non-zero `instructions_chars`
• endpoint parsed from `AZURE_FOUNDRY_CONNECTION_STRING`

B6. Create the managed agent in Foundry

```
cd copilot_agent
python deploy_to_foundry.py -v
# optional:
python deploy_to_foundry.py --agent-name val-copilot-prod
```

What the script does

1. Load credentials via `DefaultAzureCredential`
2. Parse `AZURE_FOUNDRY_CONNECTION_STRING` → `AIProjectClient`
3. Import MCP tools from `mcp_server/server.py`
4. Wrap tools with `FunctionTool` + `ToolSet`
5. Call `project_client.agents.create_agent(...)` with model, name, `instructions=SYSTEM_PROMPT`, toolset
6. Print JSON including `agent_id` on success

This is the step that creates the agent object inside your Foundry project. Part A only prepared the project/model; Part B registers VAL CoPilot.

B7. Verify on the Azure Foundry side (after script success)

Return to ai.azure.com → your project:

1. Open **Build** → **Agents**.
2. Confirm agent `val-copilot` (or your custom name) appears.
3. Open the agent and check:
- **Instructions** contain Comparative Analysis + Contract Lifecycle Procedures + invoice/spend OOS guardrail
- **Tools** list the seven function tools (search/profile/expiring/spend/overlaps/risk/blob)
- **Model** matches the deployment from **A3**
4. Use the **Agent playground** (if available) with demo prompts from `docs/demo_script.md`, for example:
- “Which contracts expire in 90 days?”

- “Any overlapping Microsoft contracts?”

- “Show invoice totals for Microsoft” → expect exact OOS message (no tool calls)

5. If tool calls fail in playground, verify fixture/offline flags or Fabric/Search credentials and that the host executing function tools can reach those services.

Runtime & architecture notes

The Foundry-managed agent stores **instructions + tool definitions**. Function tool

implementations are the Python MCP functions from this repo and run in the

process that handles tool calls.

• **Preferred for this POC:** `USE_OFFLINE_MOCKS=true` against `mcp_server/LinSquare_Contracts_100_Updated_30bb.json` + `agreement_9a06.json` (via `linksquares_fixtures.py`)
• **Optional live backends:** Fabric SQL (ODBC Driver 18 + ActiveDirectoryDefault) and Azure AI Search
• **Hard OOS:** invoice/spend-actuals questions must return the exact POC OOS string and must not call spend tools as if they were invoice APIs
• **Compare hard-stop:** if any compare side is unresolved, return only: `The contract information requested for the comparison is not available at the moment` (no table, no recommendation)
• **Persona memory:** local POC uses `memory/store.py` + Flask `/api/memory/*` + Streamlit Saved searches; not provisioned by the Foundry script
• Architecture / process flow: `docs/VAL_CoPilot_Architecture_and_Process_Flow.pdf` (and `.pptx`)
• POC change log / scripts: `docs/VAL_CoPilot_POC_Changes_and_Scripts.md`

Cognitive procedures encoded in SYSTEM_PROMPT

• Comparative Analysis & Decision Framework → `## Recommendation` (**only** when `compare_contracts` succeeds)
• Compare hard-stop → exact unavailable message only (no default pair)
• Red-Flag Compliance Audits → `## Red-Flag Compliance Audit`
• Dynamic Counter-Clause Drafting → `## Dynamic Counter-Clause Drafting`
• Financial Exposure Projections → `## Financial Exposure Projection`
• Proactive Renewal Strategy Sheets → `## Proactive Renewal Strategy Sheet`
• Persona memory recall guidance (served by local Flask/SQLite path today)
• Invoice / spend-actuals → hard OOS (separate data-linkage POC)

Troubleshooting

Symptom	Likely cause / fix
No Project endpoint in portal	Hub-only / classic project — create a **new Foundry project** (A2)
Agents blade missing	Unsupported region or classic experience — change region / enable New Foundry (A4)
Missing `AZURE_FOUNDRY_CONNECTION_STRING`	Paste endpoint from A6 into root `.env`
`create_agent` auth / 403	RBAC missing — assign Azure AI Developer (A5); re-login
Wrong model / deployment not found	Deployment name mismatch — copy exact name from A3
Cannot import `ToolSet` / `FunctionTool`	Reinstall pinned `azure-ai-projects==1.0.0` and `azure-ai-agents<2`
Tool returns empty / SQL errors	Fabric/Search env or network (A7); test MCP locally
Firewall / VNet blocks	Allowlist runner IPs on Foundry + Search (A7)

Full deployment checklist

Azure Foundry side (Part A)

- ■ A1 Portal sign-in + New Foundry ON
- ■ A2 Foundry project created
- ■ A3 Model deployed (name recorded)
- ■ A4 Agents blade opens
- ■ A5 RBAC assigned
- ■ A6 Project endpoint copied
- ■ A7 Networking done (if needed)
- ■ A8 Readiness gate passed

Code / script side (Part B)

- ■ Repo cloned
- ■ Root `.env` populated from Part A values
- ■ `az login` / managed identity ready
- ■ Dependencies installed
- ■ `--dry-run` succeeds
- ■ `deploy_to_foundry.py` returns `status=created` + `agent_id`
- ■ Portal Agents view shows agent, tools, and SYSTEM_PROMPT procedures
- ■ Playground / smoke-test prompts succeed

Keep secrets out of Git; commit only `.env.example`. Script path: `copilot_agent/deploy_to_foundry.py`.
